Temco Platform

Entry points · built Nov 10, 2026

Entry points

The fork touches VS Code's source in exactly two places. Both are additive lines in lists. This is the complete list.

1. The compile list

File	`src/tsconfig.json`
Addition	`src/temco/**` added to the `include` list
Why	the config includes `./vs/**` and a few root files only. Without this line the shell is not compiled and not type-checked

2. The load list

File	`src/vs/workbench/workbench.desktop.main.ts`
Addition	one import of `../temco/temco.contribution.js`
Why	the workbench loads its parts, services and contributions through explicit imports. Code that nothing imports never runs

This is how VS Code's own features enter the workbench: a contribution file that is imported and registers itself. The menu bar is built the same way — it renders whatever is registered in the menu registry, and a feature such as the debugger adds its own top-level menu from its own contribution file. That is why Temco's menus are Temco's own code.

The layering rule under that import

Every file under `src/` is also governed by `local/code-import-patterns`: each one must fall in a zone, and the zone carries the list of what it may import. Two entries in `eslint.config.js` make the shell legal, and they are the reason the entry points above are the whole seam rather than the beginning of one:

Zone target	Allows
`src/temco/**`	`vs/base`, `vs/platform`, `vs/editor`, `vs/workbench`, `vs/nls.js`, and itself — the shell sits outside VS Code's layering, so its reach is stated once instead of inherited
`src/vs/workbench/workbench.desktop.main.ts`	its usual list, plus `src/temco/**` — the one place allowed to import the shell

Without the first entry a shell file is "not in a valid layer"; without the second the seam import violates the workbench zone. Neither is a relaxation for the rest of `src/vs/**`.

Not entry points

Not a seam	Why
`product.json`	the fork's own identity file: product name, icons, default settings, update and telemetry policy
`resources/**`	Temco's own images
`extensions/temco-*/**`	new folders; the build picks up any folder with a `package.json`
`docs/**`	Temco's own documentation
module build wiring	none needed: modules are compiled by the existing extension build, and each folder's dependencies are installed by the root install
`build/lib/extensions.ts`	touched only if a module ships a native npm module, which no planned module does

Gate exemptions, not seams

VS Code's gates encode policies for Microsoft's own code: a Microsoft copyright header, tab indentation, `nls` for every user-visible string, no synchronous file I/O. Temco's code follows its own conventions, so the same lines that exempt `extensions/temco-*` also exempt `src/temco/**`:

File	Addition
`build/filters.ts`	`'!extensions/temco-*/**'` and `'!src/temco/**'` in the unicode, indentation, copyright and formatting filters
`eslint.config.js`	one override for both paths, turning off the Microsoft header, sync-fs, unexternalized-strings, in-operator and bracket-notation rules
`.eslint-allowed-javascript-files`	`extensions/temco-ai/media/ai-panel.js` — a webview asset, not extension code
`build/gulpfile.extensions.ts`	the module's `tsconfig.json` in the compiled list
`package.json`	`check-temco` (the module guards, the dependency graph, the menu ids), run by `compile`

None of these is a seam into VS Code's behaviour: remove them and Temco's files stop passing the gates, but the application still builds and runs unchanged.

Rule

This list may not grow without a decision. An addition needs a written reason and a row above. Before adding one, the first question is whether the need can be met by a new folder instead — a module, a `product.json` field, or an extension contribution point.

After a VS Code update

1. Does `src/tsconfig.json` still include `src/temco/**`?
2. Does `workbench.desktop.main.ts` still import `temco.contribution`?
3. Does `src/temco/temco.contribution.ts` still exist at that path?

If all three hold, the shell is intact.